
LAPORAN PRAKTIKUM TEKNOLOGI IOT II

Disusun Oleh:



Nama: RAHMAT FADILLAH PUTRA
Kelas: Teknologi Rekayasa Internet (3C)
NPM: 25782079

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

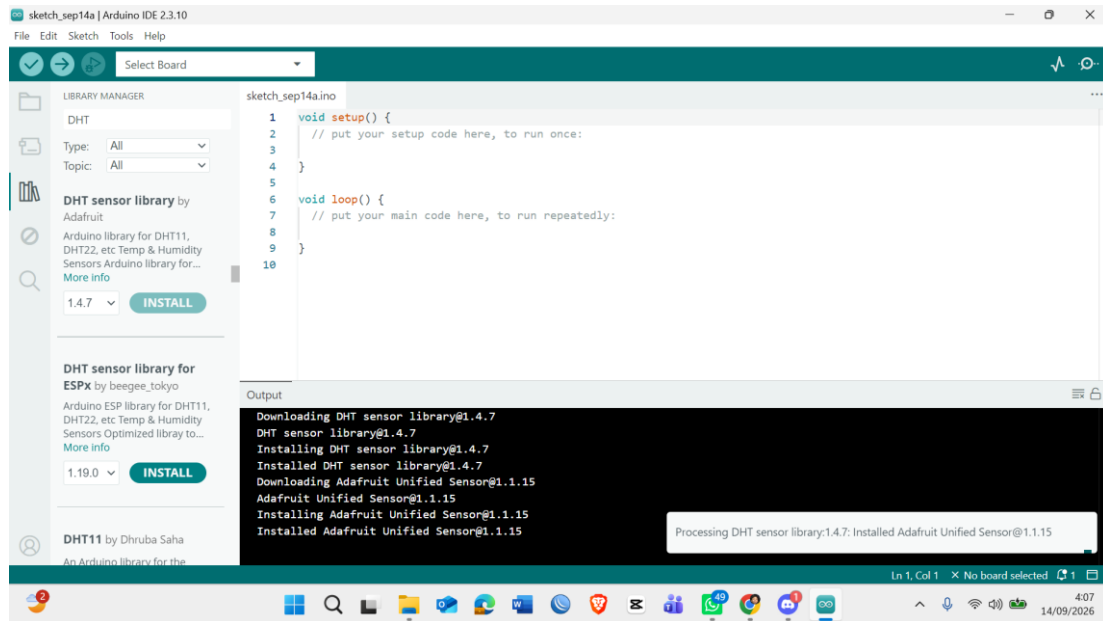
TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG 2026

1. Persiapan — Instalasi Library DHT

Sebelum melakukan pembacaan sensor DHT, terlebih dahulu dilakukan instalasi pustaka (library) 'DHT sensor library' oleh Adafruit melalui Library Manager pada Arduino IDE. Proses instalasi turut menyertakan dependensi 'Adafruit Unified Sensor' yang dibutuhkan library tersebut agar dapat beroperasi dengan baik.



Proses instalasi DHT sensor library dan Adafruit Unified Sensor pada Arduino IDE 2.3.10

2.5 Praktikum

Praktikum 1 — Pembacaan Intensitas Cahaya (LDR) via ADC

Praktikum ini bertujuan membaca nilai intensitas cahaya menggunakan sensor LDR (Light Dependent Resistor) yang dirangkai sebagai rangkaian pembagi tegangan (voltage divider), kemudian dikonversi menjadi data digital melalui fitur ADC (Analog-to-Digital Converter) pada mikrokontroler.

Code Program

```
const byte ldrPin = A0;

void setup() {
  Serial.begin(115200);
}

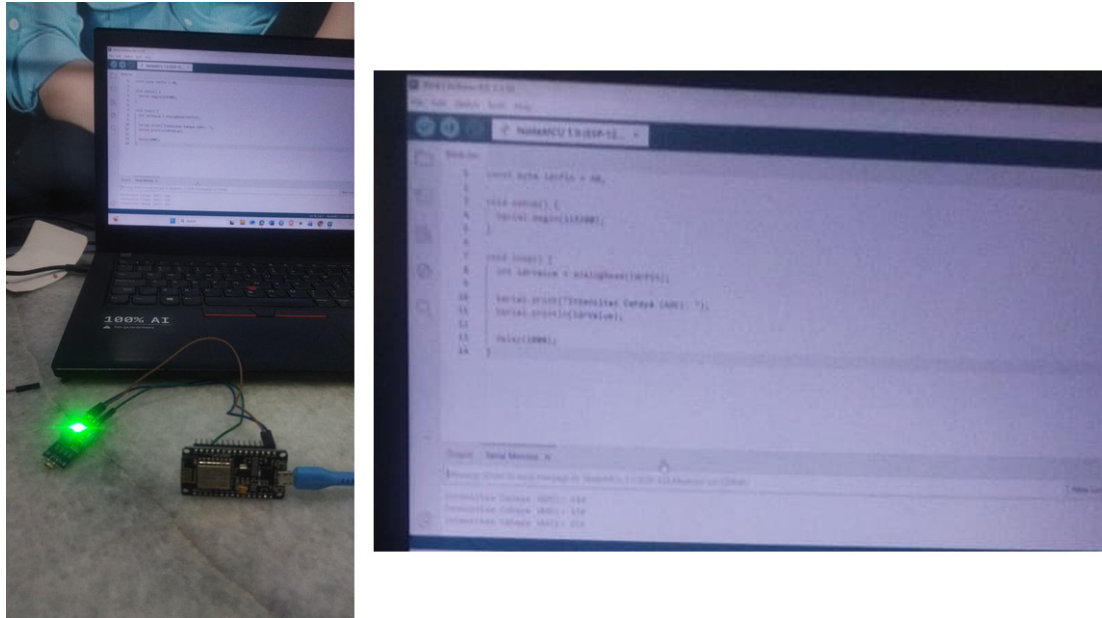
void loop() {
  int ldrValue = analogRead(ldrPin);

  Serial.print("Intensitas Cahaya (ADC): ");
  Serial.println(ldrValue);

  delay(1000);
}
```

Hasil Pengujian

Program berhasil dijalankan pada simulasi Wokwi dengan board ESP32. Nilai ADC yang terbaca berkisar antara 2777 hingga 3201, berada pada skala 12-bit (0-4095) sesuai karakteristik ADC ESP32.



Gambar 2. Rangkaian sensor LDR pada NodeMCU beserta nilai ADC pada Serial Monitor (Praktikum 1)

Praktikum 2 — Pembacaan Data Sensor Suhu & Kelembapan (DHT)

Praktikum ini bertujuan membaca data suhu dan kelembapan menggunakan sensor DHT22 melalui protokol komunikasi digital single-bus, dengan bantuan library Adafruit DHT yang telah diinstal sebelumnya.

Kode Program

```
#include <DHT.h>

const byte dhtPin = 13;
#define DHTTYPE DHT22

DHT dht(dhtPin, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
  Serial.println("Memulai Sensor Lingkungan...");
}

void loop() {
  delay(2500);

  float temp = dht.readTemperature();
```

```

float hum = dht.readHumidity();

if (isnan(temp) || isnan(hum)) {
  Serial.println("Gagal membaca data dari sensor DHT!");
  return;
}

Serial.print("Suhu: ");
Serial.print(temp);
Serial.print(" Celcius | Kelembapan: ");
Serial.print(hum);
Serial.println(" %");
}

```

Hasil Pengujian

Sensor DHT22 berhasil dibaca tanpa kegagalan (tidak ada nilai NaN). Nilai suhu dan kelembapan yang tercatat berubah-ubah sesuai simulasi (suhu 10.10°C hingga 45.50°C, kelembapan 29.00% hingga 78.50%), menunjukkan pembacaan sensor berjalan dinamis dan responsif.



Rangkaian sensor suhu dan kelembapan (DHT) pada NodeMCU (Praktikum 2)

2.6 Latihan

Latihan 1 — Pemahaman Sistem ADC

Pertanyaan: Mengapa batas maksimal nilai intensitas cahaya yang terbaca pada Serial Monitor di Praktikum 1 adalah sekitar angka 1023 (bukan 100 atau 1000)?

Jawaban

Nilai maksimal 1023 muncul karena ADC pada mikrokontroler (seperti NodeMCU ESP8266) memiliki resolusi 10-bit. Resolusi ini menentukan jumlah kombinasi biner yang dapat dihasilkan, yaitu sebanyak 2 pangkat 10 (2^{10}) = 1024 kemungkinan nilai, terhitung mulai dari 0 hingga 1023 (bukan 1 hingga 1024).

Nilai 1023 adalah representasi desimal dari nilai biner 10 digit yang seluruhnya bernilai 1 (1111111111), yaitu batas atas maksimum yang dapat direpresentasikan oleh 10 bit. Angka ini bukan kelipatan bulat seperti 100 atau 1000 karena sistem bilangan yang mendasarinya adalah biner (basis 2), bukan desimal (basis 10). Semakin tinggi resolusi bit sebuah ADC, semakin presisi/detail representasi tegangan analog yang dapat dikonversi menjadi data digital.

Catatan: pada board ESP32 yang digunakan dalam praktikum ini, resolusi ADC yang dipakai adalah 12-bit, sehingga rentang nilainya menjadi 0-4095 (2^{12} = 4096 kemungkinan nilai).

Latihan 2 — Implementasi Algoritma Persentase Cahaya

Instruksi: Modifikasi kode pada Praktikum 1 agar program menghitung dan mencetak persentase intensitas cahaya (rentang 0% hingga 100%).

Kode Program

```
const byte ldrPin = A0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  int ldrValue = analogRead(ldrPin);
  float persenCahaya = (ldrValue / 4095.0) * 100.0;

  Serial.print("Intensitas Cahaya (ADC): ");
  Serial.println(ldrValue);
  Serial.print("Persentase Cahaya: ");
  Serial.print(persenCahaya);
  Serial.println(" %");

  delay(1000);
}
```

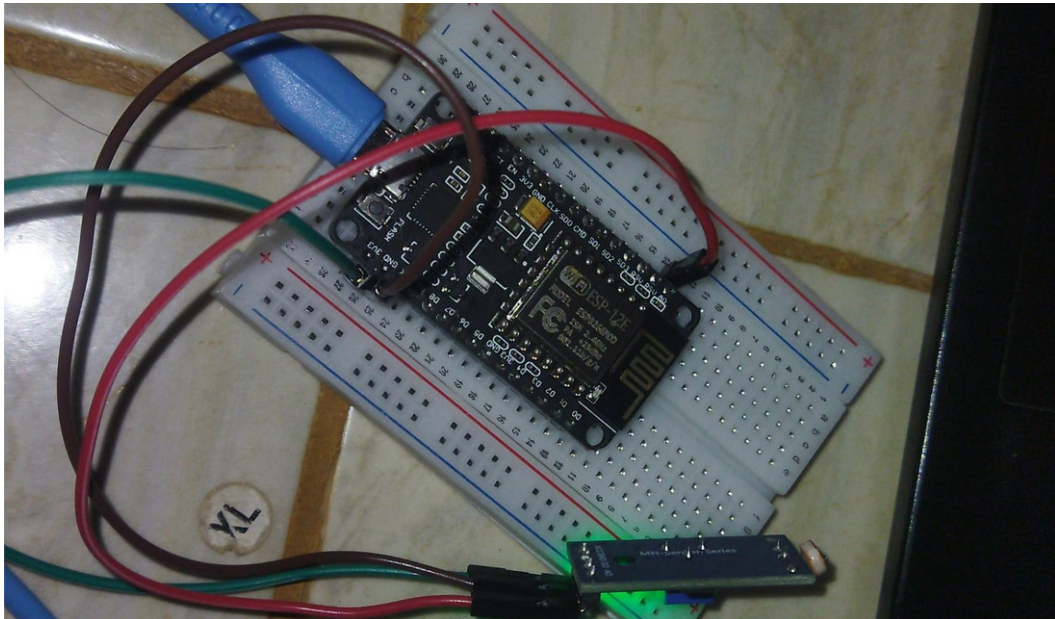
Rumus yang Digunakan

$$\text{persentase} = (\text{nilai_ADC} / 4095.0) * 100.0$$

Nilai pembagi disesuaikan menjadi 4095.0 (bukan 1023.0) karena board yang digunakan pada simulasi adalah ESP32 dengan resolusi ADC 12-bit. Angka pembagi dan pengali ditulis dalam bentuk desimal (4095.0 dan 100.0) agar operasi pembagian dilakukan sebagai floating point, bukan pembagian bilangan bulat yang akan membulatkan hasil secara tidak akurat.

Hasil Pengujian

Program berhasil menampilkan nilai ADC beserta konversi persentasenya secara bersamaan, dengan hasil yang proporsional (contoh: ADC 2007 = 49.01%, ADC 1979 = 48.33%).



Rangkaian sensor LDR pada NodeMCU untuk pembacaan persentase intensitas cahaya (Latihan 2)

Latihan 3 — Analisis Rangkaian Relay Active-Low

Pertanyaan: Jika Relay bertipe Active-Low, apa yang terjadi pada aktuator (misal: lampu ruangan) yang terhubung ke Relay jika program mengeksekusi `digitalWrite(relayPin, HIGH)`?

Jawaban

Jika relay bertipe Active-Low dan program mengeksekusi `digitalWrite(relayPin, HIGH)`, maka relay akan tetap OFF (mati), bukan menyala.

Hal ini terjadi karena pada relay active-low, logika kerjanya terbalik dari intuisi umum:

- Sinyal LOW pada pin IN → mengaktifkan optocoupler → kumparan elektromagnetik bekerja → kontak relay menutup (ON).
- Sinyal HIGH pada pin IN → optocoupler tidak aktif → kontak relay tetap terbuka (OFF).

Dengan demikian, jika seorang programmer tidak menyadari sifat active-low ini dan berasumsi bahwa HIGH selalu berarti ON (seperti logika LED pada umumnya), maka aktuator (misalnya lampu ruangan) justru tidak akan pernah menyala meskipun kode terlihat benar secara sintaks. Hal ini menegaskan pentingnya memeriksa datasheet atau karakteristik modul relay sebelum menulis logika kontrolnya. Solusi yang tepat adalah menggunakan LOW untuk mengaktifkan relay dan HIGH untuk mematikannya pada modul yang bersifat active-low.

Latihan 4 — Pengembangan Logika LED Gelap Gulita

Instruksi: Gunakan blok kondisional if untuk menyalakan sebuah LED (D1/GPIO) HANYA JIKA sensor LDR mendeteksi lingkungan yang gelap gulita.

Kode Program

```
const byte ldrPin = A0;
const byte ledPin = 2;

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
}

void loop() {
  int ldrValue = analogRead(ldrPin);
  float persenCahaya = (ldrValue / 4095.0) * 100.0;

  if (ldrValue < 800) {
    digitalWrite(ledPin, HIGH);
    Serial.println("Gelap! LED menyala.");
  } else {
    digitalWrite(ledPin, LOW);
    Serial.println("Cukup terang. LED mati.");
  }

  Serial.print("Intensitas Cahaya (ADC): ");
  Serial.println(ldrValue);
  Serial.print("Persentase Cahaya: ");
  Serial.print(persenCahaya);
  Serial.println(" %");

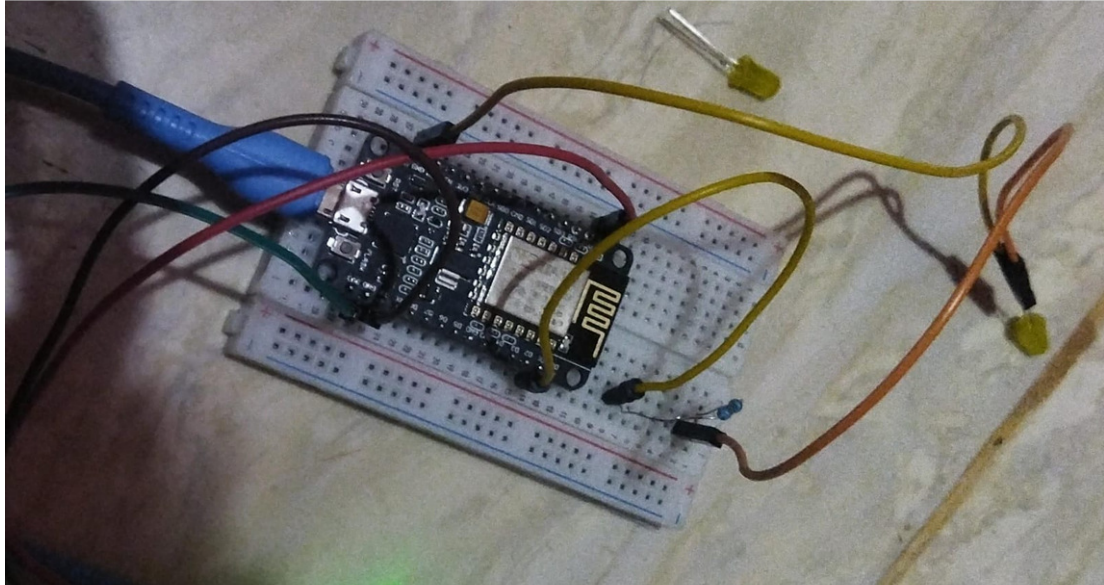
  delay(1000);
}
```

Catatan Penyesuaian Ambang Batas

Ambang batas gelap yang semula 200 (basis ADC 10-bit sesuai modul asli) dikonversi menjadi 800 (basis ADC 12-bit ESP32) menggunakan perhitungan proporsional: $200/1023 \times 4095 \approx 800$, agar logika kondisi tetap proporsional terhadap rentang sensor ADC yang sebenarnya digunakan pada board ESP32.

Hasil Pengujian

Pengujian dilakukan pada kondisi cahaya cukup terang (nilai ADC 2008-2082, persentase 49-51%), dan LED berhasil tetap mati sesuai logika program (status: 'Cukup terang. LED mati.'). Rangkaian LED beresistor 220 Ohm terpasang pada pin GPIO2 sesuai instruksi.



Rangkaian LED beresistor pada NodeMCU untuk kontrol LED berbasis sensor LDR (Latihan 4)

Tugas 2.7 — Sistem Otomasi Kontrol Lokal Terpadu (Smart Warehouse)

Tugas ini merancang sebuah rule engine aktuator mandiri untuk skenario Smart Warehouse, yang membaca Suhu (DHT22) dan Intensitas Cahaya (LDR) secara bersamaan, kemudian mengaktifkan Modul Relay 5V sebagai sakelar daya pendingin ruangan/pencahayaan darurat berdasarkan kondisi ambang batas.

Instruksi Perangkaian yang Diterapkan

- LDR (analog) terhubung ke pin ADC (GPIO34 pada ESP32).
- DHT22 (digital) terhubung ke GPIO27.
- Modul Relay VCC dihubungkan ke pin VIN (5V murni dari USB) — bukan ke 3V3, sesuai instruksi keamanan modul.
- Pin sinyal (IN) Relay terhubung ke GPIO26.
- LED indikator beresistor 220 Ohm terpasang pada GPIO2 sebagai indikator visual tambahan.

Kode Program

```
#include <DHT.h>

// ----- Konfigurasi Pin -----
const byte ldrPin   = 34;   // LDR - analog (ADC-capable)
const byte dhtPin   = 27;   // DHT - digital
const byte relayPin = 26;   // Relay - digital
const byte ledPin   = 2;    // LED indikator

#define DHTTYPE DHT22
DHT dht(dhtPin, DHTTYPE);

// ----- Konfigurasi Relay -----
const bool RELAY_ACTIVE_LOW = true;

// ----- Ambang Batas (Threshold) -----
const float SUHU_BATAS_ATAS = 34.0;   // Celcius
const int   LDR_BATAS_GELAP = 1200;   // ADC 12-bit

void setRelay(bool aktif) {
    if (RELAY_ACTIVE_LOW) {
        digitalWrite(relayPin, aktif ? LOW : HIGH);
    } else {
        digitalWrite(relayPin, aktif ? HIGH : LOW);
    }
}

void setup() {
    Serial.begin(115200);
    delay(2000);

    pinMode(relayPin, OUTPUT);
    pinMode(ledPin, OUTPUT);

    // Perintah mati di awal untuk keamanan
    setRelay(false);
    digitalWrite(ledPin, LOW);

    dht.begin();
    Serial.println("=== Sistem Smart Warehouse dimulai ===");
}

void loop() {
    int   ldrValue = analogRead(ldrPin);
    float suhu = dht.readTemperature();

    if (isnan(suhu)) {
        Serial.println("Gagal membaca data dari sensor DHT!");
        delay(1000);
        return;
    }

    // ----- Rule Engine (logika OR) -----
    bool kondisiBuruk = (suhu > SUHU_BATAS_ATAS) || (ldrValue < LDR_BATAS_GELAP);

    if (kondisiBuruk) {
        setRelay(true);
        digitalWrite(ledPin, HIGH);
        Serial.println("Peringatan: Aktuator Aktif!");
    } else {
        setRelay(false);
        digitalWrite(ledPin, LOW);
    }
}
```

```

    Serial.println("Kondisi Aman");
}

Serial.print("Suhu: ");
Serial.print(suhu);
Serial.print(" C | LDR (ADC): ");
Serial.println(ldrValue);

delay(1000);
}

```

Hasil Pengujian

Pada pengujian, parameter pencahayaan LDR diatur ke 500 lux melalui panel kontrol simulasi Wokwi. Relay dikonfigurasi sebagai active-low, dan LED indikator terlihat menyala (merah) pada rangkaian, menandakan kondisi 'kondisiBuruk' bernilai true — sesuai dengan logika OR yang diterapkan (salah satu dari kedua syarat, suhu tinggi ATAU cahaya gelap, sudah cukup untuk mengaktifkan aktuator).



```

1 #include <DHT.h>
2
3 #define DHTPIN D7
4 #define DHTTYPE DHT11
5
6 #define LDR_PIN A0
7 #define RELAY_PIN D6
8 #define LED_PIN D1
9
10 DHT dht(DHTPIN, DHTTYPE);
11
12 void setup() {
13   Serial.begin(115200);
14
15   pinMode(RELAY_PIN, OUTPUT);
16   pinMode(LED_PIN, OUTPUT);
17
18   digitalWrite(RELAY_PIN, HIGH);
19   digitalWrite(LED_PIN, HIGH);
20
21   dht.begin();
22
23   Serial.println("== SMART WAREHOUSE ==");
24   Serial.println("Sistem Otomasi Kontrol Lokal Terpadu");
25   Serial.println("");
26 }
27
28 void loop() {
29   // Read LDR sensor
30   int ldrValue = analogRead(LDR_PIN);
31
32   // Read DHT sensor
33   float temp = dht.temperature();
34   float hum = dht.humidity();
35
36   // Print sensor data
37   Serial.print("Suhu: ");
38   Serial.print(temp);
39   Serial.print(" C | LDR (ADC): ");
40   Serial.println(ldrValue);
41
42   // Delay
43   delay(1000);
44 }

```

Output Serial Monitor X

```

Message [Ctrl+Z] to send message to NodeMCU 1.0 (ESP-12E Module) on COM7
LDR
Kondisi Aman
Aktuator : MATI

== SMART WAREHOUSE ==
Sistem Otomasi Kontrol Lokal Terpadu

Suhu : 34.90 C
LDR : 464
Kondisi Aman
Aktuator : MATI

```

Rangkaian lengkap Sistem Smart Warehouse (LDR, DHT, Relay, LED) pada NodeMCU beserta tampilan Serial Monitor

Verifikasi Kriteria Keberhasilan

- Sensor LDR dan DHT berhasil dibaca bersamaan dalam satu loop tanpa kegagalan pembacaan (NaN) — divalidasi menggunakan fungsi isnan().
- Logika kondisi OR beroperasi dengan presisi — cukup salah satu syarat (suhu > 34°C ATAU LDR < ambang batas gelap) terpenuhi agar aktuator aktif.
- Relay dikonfigurasi menyesuaikan sifat active-low modul melalui fungsi setRelay(), memastikan logika ON/OFF bekerja selaras dengan karakteristik kelistrikan modul yang sebenarnya.

